

Node.js Interview Guide

Core Concepts • Express & MongoDB • Performance & Scaling • Coding, Debugging & HR

Node.js Core Concepts

1. What is Node.js?

Definition

Node.js is a JavaScript runtime built on Chrome's V8 engine that allows JavaScript to run on the server for building fast and scalable applications.

Example

```
node app.js
```

2. Features of Node.js

Definition

Node.js is event-driven, non-blocking, single-threaded, lightweight, and suitable for real-time applications.

Example

```
npm init -y
```

3. V8 Engine

Definition

The V8 engine compiles JavaScript directly into machine code, making Node.js execution fast.

Example

```
node -v
```

4. Event Loop

Definition

The Event Loop handles asynchronous operations by executing callbacks without blocking the main thread.

Example

```
setTimeout(()=>console.log('Done'),0);
```

5. Single Thread

Definition

Node.js executes JavaScript on a single main thread while delegating I/O operations to the system.

Example

```
console.log('Single Thread');
```

6. Blocking vs Non-Blocking I/O

Definition

Blocking waits for an operation to finish, while non-blocking continues execution and processes the result later.

Example

```
fs.readFile('a.txt',cb);
```

7. Callback

Definition

A callback is a function passed to another function and executed after an asynchronous task completes.

Example

```
fs.readFile('a.txt',(e,d)=>{});
```

8. Promise

Definition

A Promise represents the eventual success or failure of an asynchronous operation.

Example

```
fetchData().then(data=>{});
```

9. Async/Await

Definition

Async/Await provides a cleaner way to write asynchronous code using Promises.

Example

```
const users = await getUsers();
```

10. EventEmitter

Definition

EventEmitter allows objects to emit and listen for custom events.

Example

```
emitter.on('login', ()=>{});
```

11. Streams

Definition

Streams process data in chunks instead of loading the entire file into memory.

Example

```
fs.createReadStream('file.txt');
```

12. Buffer

Definition

A Buffer stores binary data directly in memory for file and network operations.

Example

```
Buffer.from('Hello');
```

13. File System Module

Definition

The fs module is used to create, read, update and delete files.

Example

```
fs.writeFile('a.txt', 'Hello', cb);
```

14. Path Module

Definition

The path module provides utilities for handling file and directory paths.

Example

```
path.join(__dirname, 'uploads');
```

15. Process Object

Definition

The process object provides information and control over the current Node.js process.

Example

```
process.env.PORT
```

16. Environment Variables

Definition

Environment variables store configuration values outside the application code.

Example

```
PORT=3000
```

17. CommonJS

Definition

CommonJS imports modules using `require()` and exports using `module.exports`.

Example

```
const fs=require('fs');
```

18. ES Modules

Definition

ES Modules use `import/export` syntax for modular JavaScript.

Example

```
import express from 'express';
```

19. npm

Definition

npm is the default package manager used to install and manage Node.js packages.

Example

```
npm install express
```

20. npx

Definition

npx executes npm packages without installing them globally.

Example

```
npx nodemon app.js
```

21. Global vs Local Packages

Definition

Global packages are available system-wide, while local packages belong only to the current project.

Example

```
npm install -g nodemon
```

22. Error Handling

Definition

Errors can be handled using `try/catch` for `async-await` or `error-first` callbacks.

Example

```
try { await api(); } catch(e){}
```

23. Worker Threads

Definition

Worker Threads execute CPU-intensive tasks in parallel without blocking the main thread.

Example

```
new Worker('./worker.js')
```

24. Cluster

Definition

The cluster module creates multiple Node.js processes to utilize multiple CPU cores.

Example

```
cluster.fork();
```

25. Timers

Definition

Node.js provides timers for delayed and repeated execution.

Example

```
setInterval(()=>{},1000);
```

Express.js, REST API, MongoDB, Mongoose, JWT

26. What is Express.js?

Definition

Express.js is a lightweight Node.js framework used to build REST APIs and web applications with routing and middleware support.

Example

```
const app=require('express')();
```

27. What is Middleware?

Definition

Middleware is a function that executes between the request and response cycle to process requests or responses.

Example

```
app.use((req,res,next)=>{next();});
```

28. Application vs Router Middleware

Definition

Application middleware runs for the app, while router middleware runs only for a specific router.

Example

```
const router=require('express').Router();
```

29. What is REST API?

Definition

A REST API follows HTTP methods to perform CRUD operations on resources using stateless communication.

Example

```
GET /users
```

30. Express Routing

Definition

Routing maps incoming URLs and HTTP methods to controller functions.

Example

```
app.get('/users', getUsers);
```

31. Request Object

Definition

The request object contains client data such as params, query, body and headers.

Example

```
req.params.id
```

32. Response Object

Definition

The response object sends data, status codes or files back to the client.

Example

```
res.status(200).json(users);
```

33. Error Middleware

Definition

Error middleware centrally handles application errors using four parameters.

Example

```
app.use((err, req, res, next) => res.status(500).json(err));
```

34. What is CORS?

Definition

CORS allows controlled access to resources from different origins.

Example

```
app.use(cors());
```

35. What is Helmet?

Definition

Helmet secures Express applications by setting various HTTP security headers.

Example

```
app.use(helmet());
```

36. What is Morgan?

Definition

Morgan logs incoming HTTP requests for debugging and monitoring.

Example

```
app.use(morgan('dev'));
```

37. What is JWT?

Definition

JWT is a compact token used for user authentication between client and server.

Example

```
jwt.sign(payload, secret);
```

38. JWT Verification

Definition

JWT verification validates the token before allowing access to protected routes.

Example

```
jwt.verify(token, secret);
```

39. What is bcrypt?

Definition

bcrypt securely hashes passwords before storing them in the database.

Example

```
bcrypt.hash(password, 10);
```

40. What is Multer?

Definition

Multer is middleware used to upload files in Express applications.

Example

```
upload.single('image')
```

41. What is MongoDB?

Definition

MongoDB is a NoSQL document database that stores data as flexible BSON documents.

Example

```
use('mydb');
```

42. What is Mongoose?

Definition

Mongoose is an ODM library that provides schemas, validation and models for MongoDB.

Example

```
mongoose.model('User', schema);
```

43. Schema vs Model

Definition

A Schema defines document structure, while a Model performs database operations.

Example

```
const User=mongoose.model('User',userSchema);
```

44. CRUD with Mongoose

Definition

Mongoose provides methods to create, read, update and delete documents.

Example

```
User.find(); User.create(data);
```

45. Aggregation

Definition

Aggregation processes data through pipeline stages for reporting and analytics.

Example

```
User.aggregate([{$match:{}}]);
```

Performance, Streams, Scaling & Advanced Concepts

46. Event Loop Phases

Definition

The Event Loop processes asynchronous callbacks through phases like timers, pending callbacks, poll, check, and close callbacks.

Example

```
setTimeout(()=>{},0); setImmediate(()=>{});
```

47. Streams

Definition

Streams process data in chunks, reducing memory usage for large files.

Example

```
fs.createReadStream('large.txt').pipe(res);
```

48. Readable vs Writable Streams

Definition

Readable streams consume data, while writable streams receive and store data.

Example

```
fs.createReadStream('a.txt');
```

49. Pipe()

Definition

pipe() transfers data directly from one stream to another without buffering the entire file.

Example

```
readStream.pipe(writeStream);
```

50. Buffer

Definition

Buffers store raw binary data and are commonly used for files, images, and network packets.

Example

```
Buffer.from('Hello');
```

51. Child Process

Definition

Child processes execute external commands or scripts without blocking the main process.

Example

```
exec('dir', (e, out) => {});
```

52. Worker Threads

Definition

Worker Threads run CPU-intensive tasks in parallel to improve application responsiveness.

Example

```
new Worker('./worker.js');
```

53. Cluster

Definition

Cluster creates multiple Node.js processes to utilize all CPU cores.

Example

```
cluster.fork();
```

54. Redis

Definition

Redis is an in-memory data store used for caching, sessions, and fast lookups.

Example

```
redis.set('user', json);
```

55. Caching

Definition

Caching stores frequently used data to reduce database calls and improve response time.

Example

```
redis.get('products');
```

56. Rate Limiting

Definition

Rate limiting restricts repeated client requests to prevent abuse and protect APIs.

Example

```
app.use(rateLimit({windowMs:60000,max:100}));
```

57. Socket.IO

Definition

Socket.IO enables real-time bidirectional communication between clients and servers.

Example

```
io.on('connection',socket=>{});
```

58. PM2

Definition

PM2 is a production process manager for running, monitoring, and restarting Node.js applications.

Example

```
pm2 start app.js
```

59. Docker

Definition

Docker packages an application and its dependencies into a portable container.

Example

```
docker build -t node-app .
```

60. Microservices

Definition

Microservices split an application into independent services that communicate through APIs or messaging.

Example

```
User Service -> Order Service -> Payment Service
```

Coding Questions, Debugging Scenarios, Project Discussion and HR

61. How do you create a basic CRUD API?

Definition

A CRUD API provides Create, Read, Update, and Delete operations using HTTP methods such as POST, GET, PUT or PATCH, and DELETE.

Example

```
app.post('/users', createUser);
app.get('/users', getUsers);
app.put('/users/:id', updateUser);
app.delete('/users/:id', deleteUser);
```


62. How do you create a login API?

Definition

A login API validates the user's credentials, compares the hashed password, and returns an access token after successful authentication.

Example

```
const user = await User.findOne({ email });
const valid = await bcrypt.compare(password, user.password);
const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET);
```

63. What is the JWT authentication flow?

Definition

In JWT authentication, the server creates a token after login, the client sends it in the Authorization header, and middleware verifies it before protected routes are executed.

Example

```
const token = req.headers.authorization?.split(' ')[1];
const payload = jwt.verify(token, process.env.JWT_SECRET);
req.user = payload;
```

64. What is a refresh-token flow?

Definition

A refresh token is used to generate a new short-lived access token when the existing access token expires, without requiring the user to log in again.

Example

```
app.post('/auth/refresh', (req, res) => {
  const payload = jwt.verify(req.body.refreshToken, REFRESH_SECRET);
  const accessToken = jwt.sign({ id: payload.id }, ACCESS_SECRET);
  res.json({ accessToken });
});
```

65. How do you implement role-based authorization?

Definition

Role-based authorization checks the authenticated user's role before allowing access to specific routes or operations.

Example

```
const allowRoles = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) return res.sendStatus(403);
  next();
};
```

66. How do you implement pagination?

Definition

Pagination limits the number of records returned by calculating skip and limit values from the requested page and page size.

Example

```
const page = Number(req.query.page || 1);
const limit = Number(req.query.limit || 10);
const users = await User.find().skip((page - 1) * limit).limit(limit);
```

67. How do you create a search API?

Definition

A search API filters records based on user input, commonly using regular expressions, indexes, or database text search.

Example

```
const users = await User.find({
  name: { $regex: req.query.q, $options: 'i' }
});
```

68. How do you upload files in Node.js?

Definition

File uploads are commonly handled using Multer middleware, which processes multipart form data and stores the uploaded file.

Example

```
const upload = multer({ dest: 'uploads/' });
app.post('/upload', upload.single('file'), (req, res) => {
  res.json({ file: req.file });
});
```

69. How do you validate request data?

Definition

Request validation checks the body, params, and query values before business logic runs, preventing invalid or unsafe data from reaching the database.

Example

```
const schema = Joi.object({
  email: Joi.string().email().required(),
  password: Joi.string().min(8).required()
});
const { error } = schema.validate(req.body);
```

70. How do you handle errors globally?

Definition

Global error middleware catches application errors in one place and returns consistent status codes and response structures.

Example

```
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({
    message: err.message || 'Internal server error'
  });
});
```

71. How do you handle async route errors?

Definition

Async route errors can be forwarded to global error middleware by wrapping controllers and passing rejected promises to next().

Example

```
const asyncHandler = fn => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

app.get('/users', asyncHandler(getUsers));
```

72. How do you prevent duplicate database records?

Definition

Duplicate records are prevented using unique database indexes and by handling duplicate-key errors in the application.

Example

```
const userSchema = new mongoose.Schema({
  email: { type: String, unique: true }
});
```

73. How do you secure passwords?

Definition

Passwords should be hashed using a strong algorithm such as bcrypt before storage and compared using bcrypt during login.

Example

```
user.password = await bcrypt.hash(password, 12);
const valid = await bcrypt.compare(inputPassword, user.password);
```

74. How do you protect an API from too many requests?

Definition

Rate limiting restricts how many requests a client can make within a specific time window, reducing abuse and denial-of-service risk.

Example

```
app.use(rateLimit({
  windowMs: 60 * 1000,
  max: 100
}));
```

75. How do you enable API versioning?

Definition

API versioning maintains backward compatibility by separating endpoint versions when response formats or business rules change.

Example

```
app.use('/api/v1/users', userRoutesV1);
app.use('/api/v2/users', userRoutesV2);
```

76. How do you implement caching?

Definition

Caching stores frequently requested data in a fast store such as Redis to reduce database load and improve response time.

Example

```
const cached = await redis.get('products');
if (cached) return res.json(JSON.parse(cached));
await redis.setEx('products', 300, JSON.stringify(products));
```

77. Why might an API be called twice?

Definition

An API may run twice because of duplicate frontend requests, repeated event handlers, retry logic, middleware calling next() incorrectly, or duplicate route registration.

Example

```
app.get('/users', requestLogger, getUsers);
// Ensure middleware calls next() only once.
```

78. How do you debug a slow API?

Definition

A slow API is investigated by measuring request time, checking database queries and indexes, reviewing external calls, and profiling CPU or memory usage.

Example

```
console.time('getUsers');
const users = await User.find({ status: 'active' }).lean();
console.timeEnd('getUsers');
```

79. How do you fix high memory usage?

Definition

High memory usage is reduced by removing unused references, clearing timers, avoiding unbounded caches, streaming large files, and profiling heap usage.

Example

```
const stream = fs.createReadStream('large-file.csv');
stream.pipe(res);
```

80. How should a Node.js project be structured?

Definition

A maintainable Node.js project separates routes, controllers, services, models, middleware, configuration, utilities, and tests.

Example

```
src/
routes/
controllers/
services/
models/
middleware/
config/
tests/
```

81. What is the controller-service-repository pattern?

Definition

This pattern separates HTTP handling, business logic, and database access so that each layer has one clear responsibility.

Example

```
router -> controller -> service -> repository -> database
```

82. How do you explain your Node.js project?

Definition

A strong project explanation covers the business purpose, architecture, APIs, database, authentication, responsibilities, challenges, and performance improvements.

Example

```
Project: Order Management API
Stack: Node.js, Express, MongoDB, Redis, JWT
Role: API development, authentication, caching, deployment
```

83. What production issue did you solve?

Definition

An effective answer describes the issue, root cause, solution, and measurable result, such as reducing response time through indexing and caching.

Example

Problem: Product API took 3 seconds
Solution: Added MongoDB index and Redis cache
Result: Reduced average response time below 300 ms

84. How do you test Node.js APIs?

Definition

Node.js APIs are tested using unit and integration tests with tools such as Jest and Supertest, while external services are mocked when required.

Example

```
await request(app)
  .get('/api/users')
  .expect(200);
```

85. How do you introduce yourself for a Node.js interview?

Definition

A concise introduction should mention your total experience, Node.js skills, databases, projects, current responsibilities, and one strong technical achievement.

Example

I have strong experience building Node.js and Express APIs with MongoDB, JWT authentication, Redis caching, and production deployments.